MTE Vault Add-On Developer's Guide

MTE Version 2.1

Table of Contents

1 Introduction
- 1.1 Server-Controlled Entropy With PIN Nonce
- 1.2 Device Owner Entropy And Nonce

2 Terms and Abbreviations

3 Prerequisites

4 Caveats

5 Architecture

6 SDK Structure

7 Options

8 Status

9 Java Language Interface
- 9.1 Overview
- 9.2 Files
- 9.3 Types
  - 9.3.1 Classes
    - 9.3.1.1 `MteVault`
    - 9.3.1.2 `MteVault.VaultException`
- 9.4 Functions
  - 9.4.1 `MteVault` Constructor (default MKE) (Java)
  - 9.4.2 `MteVault` Constructor (provided MKE) (Java)
  - 9.4.3 `MteVault.getEncoder` (Java)
  - 9.4.4 `MteVault.getDecoder` (Java)
  - 9.4.5 `MteVault.initVault` (Java)
  - 9.4.6 `MteVault.readData` (Java)
  - 9.4.7 `MteVault.readString` (Java)
  - 9.4.8 `MteVault.write` (data, default to storage) (Java)
  - 9.4.9 `MteVault.write` (data) (Java)
  - 9.4.10 `MteVault.write` (string, default to storage) (Java)
  - 9.4.11 `MteVault.write` (string) (Java)
  - 9.4.12 `MteVault.remove` (Java)
  - 9.4.13 `MteVault.removeVault` (Java)
  - 9.4.14 `MteVault.dirExists` (Java)
  - 9.4.15 `MteVault.fileExists` (Java)
  - 9.4.16 `MteVault.listFiles` (Java)
  - 9.4.17 `MteVault.createDir` (Java)
  - 9.4.18 `MteVault.readFile` (Java)
  - 9.4.19 `MteVault.writeFile` (Java)
  - 9.4.20 `MteVault.removeDir` (Java)
  - 9.4.21 `MteVault.removeFile` (Java)
  - 9.4.22 `MteVault.getTimestamp` (Java)

10 Swift Language Interface

10.1 Overview
10.2 Files
10.3 Types
10.3.1 Classes
10.3.1.1 `MteVault`
10.4 Functions
10.4.1 `MteMkeDec` Initializer (default MKE) (Swift)
10.4.2 `MteMkeDec` Initializer (provided MKE) (Swift)
10.4.3 `MteVault.getEncoder` (Swift)
10.4.4 `MteVault.getDecoder` (Swift)
10.4.5 `MteVault.initVault` (Swift)
10.4.6 `MteVault.readData` (Swift)
10.4.7 `MteVault.readString` (Swift)
10.4.8 `MteVault.write` (data) (Swift)
10.4.9 `MteVault.write` (string) (Swift)
10.4.10 `MteVault.remove` (Swift)
10.4.11 `MteVault.removeVault` (Swift)
10.4.12 `MteVault.dirExists` (Swift)
10.4.13 `MteVault.fileExists` (Swift)
10.4.14 `MteVault.listFiles` (Swift)
10.4.15 `MteVault.createDir` (Swift)
10.4.16 `MteVault.readFile` (Swift)
10.4.17 `MteVault.writeFile` (Swift)
10.4.18 `MteVault.removeDir` (Swift)
10.4.19 `MteVault.removeFile` (Swift)
10.4.20 `MteVault.getTimestamp` (Swift)
11 **Contact Eclypses**

1 Introduction

The MTE Vault Add-On uses the MTE Managed-Key Encryption Add-On to create a secure vault in memory or on storage. This vault can hold files securely without depending on the operating system or other mechanisms which may be prone to security flaws which are exploited by attackers. Due to the design of MTE, the information needed to unlock the vault can be split and stored in separate locations, none of which are on the device where the vault exists. This design protects the vault against typical attacks on device operating systems.

The MTE Vault Add-On uses four pieces of information to encrypt files: entropy, nonce, the filename, and a timestamp. Without all four pieces of information, the encrypted files cannot be decrypted. The timestamp is stored with the file, and its only purpose is to ensure that the same file contents and name will encrypt differently each time they are encrypted. The filename is used as the personalization string, again to ensure that the same file contents encrypt differently in different files. The biggest keys to the security are the entropy and nonce. Refer to the *MTE Core Developer's Guide* for more information about the entropy, nonce, and personalization string.

The MTE Vault Add-On is written to support both memory-only and permanent storage files. The memory-only files will disappear when the application exits or the vault object is destroyed. The permanent storage files are persisted until explicitly removed. The memory-only feature makes storing small pieces of sensitive data securely extremely efficient and easy to use, so applications can avoid storing sensitive data in the clear in memory, helping to reduce the success of certain attack vectors.

By default, the MTE Vault Add-On's permanent storage uses the default filesystem API of the device. The classes are written with abstracted input/output (I/O), so it is possible to provide alternate storage APIs. Using this capability, the files could be written anywhere the developer chooses, including on remote devices or in databases.

By default the MTE Vault Add-On acquires a timestamp using a basic API. The classes are written with abstracted timestamp acquisition, so it is possible to provide an alternate timestamp source. A higher resolution timestamp source can be beneficial if files are written quickly because it introduces more variation.

There are many ways to design systems to use the MTE Vault Add-On to enhance security. All examples show uses of the vault that would prevent a compromised device from compromising the secured data. Even if the operating system is compromised, the data is protected because nothing on the device has all of the information needed to decrypt the files. The following subsections describe some examples.

1.1 Server-Controlled Entropy With PIN Nonce

In this example, the application communicates with a remote server. The application may be storing sensitive data on the device which should only be unlocked and used by the application.

To create the vault, the server generates some entropy and sends it to the application (via secure means like PKI or using MTE) as well as storing it for later use. The application asks the user for a PIN (to be the nonce). This PIN does not have to be particularly strong because the security is ensured by the entropy, so the user experience is a good one. The PIN simply ensures that the entropy alone, if somehow captured, would not be enough to unlock the Vault Add-On. The vault can now be created and files can be stored.

Once the vault is created, later uses of the application will need to contact the server to get the saved entropy (securely) and prompt for the user's PIN in order to initialize and decrypt the persisted files successfully.

Variations of this example could allow for complete server control of the files by the server providing the nonce as well, which could be useful in a DRM scenario where the application wants complete control over the usability of the data, regardless of the user.

1.2 Device Owner Entropy And Nonce

In this example, the application may not communicate with any server. The application may be storing personal data like photos which should only be unlocked when the device owner wants to use them.

To create the vault, the application needs to acquire some entropy. This may be in the form of a password or biometrics or something similar. The application may also ask for a PIN (to be the nonce) for an extra level of security. The entropy and nonce must not be stored on the device. The vault can now be created and files can be stored.

Once the vault is created, later uses of the application will need to prompt again for the entropy and nonce sources in order to initialize and decrypt the persisted files successfully.

2 Terms and Abbreviations

Refer to the *MTE Core Developer's Guide* terms and abbreviations.

3 Prerequisites

Refer to the *MTE Core Developer's Guide* prerequisites.

4 Caveats

Refer to the *MTE Core Developer's Guide* caveats.

5 Architecture

Refer to the *MTE Core Developer's Guide* architecture.

6 SDK Structure

Refer to the *MTE Core Developer's Guide* SDK structure.

7 Options

Refer to the *MTE Core Developer's Guide* and *MTE Managed-Key Encryption Add-On Developer's Guide* options. If you provide the MKE encoder and decoder to the vault, the options must be the same for the encoder and decoder.

8 Status

Refer to the *MTE Core Developer's Guide* status.

9 Java Language Interface

9.1 Overview

The basic flow is to create the vault, initialize it, then use it to read and write files. Files in the vault and the vault itself can be removed. The vault class can be derived from to change the storage and timestamp behavior.

To create, use an `MteVault` constructor.

To initialize, call `MteVault.initVault()`.

To read files, use `MteVault.readData()` or `MteVault.readString()`. To write files, use `MteVault.write()`.

To remove files from the vault, use `MteVault.remove()`.

To remove the entire vault, use `MteVault.removeVault()`.

To change the storage and/or timestamp behavior, override the protected methods.

9.2 Files

The MTE Vault Add-On uses the MTE Managed-Key Encryption Add-On source files documented in the *MTE Manged-Key Encryption Add-On Developer's Guide*, as well as the following:

File	Description
MteVault.java	The MteVault class.

9.3 Types

9.3.1 Classes

9.3.1.1 ***`MteVault`***

Defines vault methods.

9.3.1.2 ***`MteVault.VaultException`***

Defines MTE exceptions thrown by vault methods.

9.4 Functions

9.4.1 `MteVault` Constructor (default MKE) (Java)

```
public MteVault(String vaultDir)
```

Constructor using default MKE encoder and decoder and the given vault directory.

`vaultDir`: the vault directory.

9.4.2 `MteVault` Constructor (provided MKE) (Java)

```
public MteVault(MteMkeEnc enc, MteMkeDec dec, String vaultDir)
```

Constructor using provided MKE encoder and decoder and the given vault directory. The encoder and decoder provided to this object cannot be used outside this object as this object will change their states. The encoder and decoder must have been created with matching options.

`enc`: the MKE encoder.
`dec`: the MKE decoder.
`vaultDir`: the vault directory.

9.4.3 `MteVault.getEncoder` (Java)

```
public MteMkeEnc getEncoder()
```

Returns the MKE encoder in use. This should only be used for information.

9.4.4 `MteVault.getDecoder` (Java)

```
public MteMkeDec getDecoder()
```

Returns the MKE decoder in use. This should only be used for information.

9.4.5 `MteVault.initVault` (Java)

```
public void initVault(byte[] entropy, long nonce) throws IOException
```

Initializes the vault with the entropy and nonce to use. Throws an exception if the vault cannot be created.

`entropy`: the entropy to use.
`nonce`: the nonce to use.

9.4.6 `MteVault.readData` (Java)

```
public byte[] readData(String name) throws IOException, VaultException
```

Read from storage or memory as data. If the same `name` exists in memory and on storage, the memory version is read. Throws an exception on I/O error or MTE error.

`name`: the filename.

9.4.7 `MteVault.readString` (Java)

```
public String readString(String name) throws IOException, VaultException
```

Read from storage or memory as a string. If the same `name` exists in memory and on storage, the memory version is read. Throws an exception on I/O error or MTE error.

`name`: the filename.

9.4.8 `MteVault.write` (data, default to storage) (Java)

```
public void write(String name, byte[] data) throws IOException, VaultException
```

Calls `write(name, data, false)`.

9.4.9 `MteVault.write` (data) (Java)

```
public void write(String name, byte[] data, boolean toMemory) throws IOException,
VaultException
```

Write the given data to storage or memory. If the `name` matches the name of a previously written file, this will overwrite it. If `toMemory` is `true`, the data is saved to memory and not written to permanent storage; there are no restrictions on the contents of the `name` argument. If `toMemory` is `false`, the data is saved to permanent storage in the vault directory set in the constructor; the `name` argument must be a valid filename. Throws an exception on I/O error or MTE error.

`name`: the filename.
`data`: the data to store.
`toMemory`: `true` to save to memory or `false` to save to storage.

9.4.10 `MteVault.write` (string, default to storage) (Java)

```
public void write(String name, String str) throws IOException, VaultException
```

Calls `write(name, str, false)`.

9.4.11 `MteVault.write` (string) (Java)

```
public void write(String name, String str, boolean toMemory) throws IOException,
VaultException
```

Write the given string to storage or memory. If the `name` matches the name of a previously written file, this will overwrite it. If `toMemory` is `true`, the string is saved to memory and not written to permanent storage; there are no restrictions on the contents of the `name` argument. If `toMemory` is `false`, the string is saved to permanent storage in the vault directory set in the constructor; the `name` argument must be a valid filename. Throws an exception on I/O error or MTE error.

`name`: the filename.
`str`: the string to store.
`toMemory`: `true` to save to memory or `false` to save to storage.

9.4.12 `MteVault.remove` (Java)

```
public void remove(String name) throws IOException
```

Removes a vault item. If the same name exists in memory and on storage, the memory version is removed. It is not an error to remove an item that does not exist. An exception is thrown if the file exists and cannot be removed.

`name`: the filename.

9.4.13 `MteVault.removeVault` (Java)

```
public void removeVault() throws IOException
```

Removes the vault. All memory and storage items are removed. This object is not usable until a new call to `initVault()`. It is not an error to remove a vault that does not exist. An exception is thrown if any file in the vault cannot be removed.

9.4.14 `MteVault.dirExists` (Java)

```
protected boolean dirExists(String dir)
```

Returns `true` if the directory exists, `false` if not. The `java.io.File` API is used. Override to provide an alternate implementation.

`dir`: the directory name.

9.4.15 `MteVault.fileExists` (Java)

```
protected boolean fileExists(String dir, String file)
```

Returns `true` if the `file` exists in `dir`, `false` if not. The `java.io.File` API is used. Override to provide an alternate implementation.

`dir`: the directory name.
`file`: the filename.

9.4.16 `MteVault.listFiles` (Java)

```
protected String[] listFiles(String dir) throws IOException
```

Returns a list of file basenames in a directory. Throws an exception on failure. The `java.io.File` API is used. Override to provide an alternate implementation.

`dir`: the directory name.

9.4.17 `MteVault.createDir` (Java)

```
protected void createDir(String dir) throws IOException
```

Creates a directory, including any intermediate directories as necessary. Throws an exception on failure. The `java.io.File` API is used. Override to provide an alternate implementation.

`dir`: the directory name.

9.4.18 `MteVault.readFile` (Java)

```
protected byte[] readFile(String dir, String file) throws IOException
```

Reads a file. Returns the file contents. Throws an exception on failure. The `java.io.File` and `java.io.FileInputStream` APIs are used. Override to provide an alternate implementation.

`dir`: the directory name.
`file`: the filename.

9.4.19 `MteVault.writeFile` (Java)

```
protected void writeFile(String dir, String file, byte[] contents) throws
IOException
```

Writes a file. Throws an exception on failure. The `java.io.File` and `java.io.FileOutputStream` APIs are used. Override to provide an alternate implementation.

`dir`: the directory name.
`file`: the filename.
`contents`: the contents of the file to write.

9.4.20 `MteVault.removeDir` (Java)

```
protected void removeDir(String dir) throws IOException
```

Removes a directory. Throws an exception on failure. The `java.io.File` API is used. Override to provide an alternate implementation.

`dir`: the directory name.

9.4.21 `MteVault.removeFile` (Java)

```
protected void removeFile(String dir, String file) throws IOException
```

Removes a file. Throws an exception on failure. The `java.io.File` API is used. Override to provide an alternate implementation.

`dir`: the directory name.
`file`: the filename.

9.4.22 `MteVault.getTimestamp` (Java)

```
protected long getTimestamp()
```

Returns the timestamp, byte swapped to increase the entropy in the upper bytes. The `java.lang.System.currentTimeMillis()` function is used. Override to provide an alternate implementation. It is recommended to byte swap the value so the most-changing part of the timestamp is in the upper bytes of the `long`.

10 Swift Language Interface

10.1 Overview

The basic flow is to create the vault, initialize it, then use it to read and write files. Files in the vault and the vault itself can be removed. The vault class can be derived from to change the storage and timestamp behavior.

To create, use an `MteVault` initializer.

To initialize, call `MteVault.initVault()`.

To read files, use `MteVault.readData()` or `MteVault.readString()`. To write files, use `MteVault.write()`.

To remove files from the vault, use `MteVault.remove()`.

To remove the entire vault, use `MteVault.removeVault()`.

To change the storage and/or timestamp behavior, override the internal methods.

10.2 Files

The MTE Vault Add-On uses the MTE Managed-Key Encryption Add-On source files documented in the *MTE Manged-Key Encryption Add-On Developer's Guide*, as well as the following:

File	Description
MteVault.swift	The MteVault class.

10.3 Types

10.3.1 Classes

10.3.1.1 ***`MteVault`***

Defines vault methods.

10.4 Functions

10.4.1 `MteMkeDec` Initializer (default MKE) (Swift)

```
public convenience init(_ vaultDir: String) throws
```

Initializer using default MKE encoder and decoder and the given vault directory.

`vaultDir`: the vault directory.

10.4.2 `MteMkeDec` Initializer (provided MKE) (Swift)

```
public init(_ enc: MteMkeEnc, _ dec: MteMkeDec, _ vaultDir: String) throws
```

Initializer using provided MKE encoder and decoder and the given vault directory. The encoder and decoder provided to this object cannot be used outside this object as this object will change their states. The encoder and decoder must have been created with matching options.

`enc`: the MKE encoder.
`dec`: the MKE decoder.
`vaultDir`: the vault directory.

10.4.3 `MteVault.getEncoder` (Swift)

```
public func getEncoder() -> MteMkeEnc
```

Returns the MKE encoder in use. This should only be used for information.

10.4.4 `MteVault.getDecoder` (Swift)

```
public func getDecoder() -> MteMkeDec
```

Returns the MKE decoder in use. This should only be used for information.

10.4.5 `MteVault.initVault` (Swift)

```
public func initVault(_ entropy: [UInt8], _ nonce: UInt64) throws
```

Initializes the vault with the entropy and nonce to use. Throws an exception if the vault cannot be created.

`entropy`: the entropy to use.
`nonce`: the nonce to use.

10.4.6 `MteVault.readData` (Swift)

```
public func readData(_ name: String) throws -> ArraySlice<UInt8>
```

Read from storage or memory as data. If the same `name` exists in memory and on storage, the memory version is read. Throws an exception on I/O error.

`name`: the filename.

10.4.7 `MteVault.readString` (Swift)

```
public func readString(_ name: String) throws -> String
```

Read from storage or memory as a string. If the same `name` exists in memory and on storage, the memory version is read. Throws an exception on I/O error.

`name`: the filename.

10.4.8 `MteVault.write` (data) (Swift)

```
public func write(_ name: String, _ data: [UInt8], toMemory: Bool = false) throws
```

Write the given data to storage or memory. If the `name` matches the name of a previously written file, this will overwrite it. If `toMemory` is `true`, the data is saved to memory and not written to permanent storage; there are no restrictions on the contents of the `name` argument. If `toMemory` is `false`, the data is saved to permanent storage in the vault directory set in the initializer; the `name` argument must be a valid filename. Throws an exception on I/O error.

`name`: the filename.
`data`: the data to store.
`toMemory`: `true` to save to memory or `false` to save to storage.

10.4.9 `MteVault.write` (string) (Swift)

```
public func write(_ name: String, _ str: String, toMemory: Bool = false) throws
```

Write the given string to storage or memory. If the `name` matches the name of a previously written file, this will overwrite it. If `toMemory` is `true`, the string is saved to memory and not written to permanent storage; there are no restrictions on the contents of the `name` argument. If `toMemory` is `false`, the string is saved to permanent storage in the vault directory set in the initializer; the `name` argument must be a valid filename. Throws an exception on I/O error.

`name`: the filename.
`str`: the string to store.
`toMemory`: `true` to save to memory or `false` to save to storage.

10.4.10 `MteVault.remove` (Swift)

```
public func remove(_ name: String) throws
```

Removes a vault item. If the same name exists in memory and on storage, the memory version is removed. It is not an error to remove an item that does not exist. An exception is thrown if the file exists and cannot be removed.

`name`: the filename.

10.4.11 `MteVault.removeVault` (Swift)

```
public func removeVault() throws
```

Removes the vault. All memory and storage items are removed. This object is not usable until a new call to `initVault()`. It is not an error to remove a vault that does not exist. An exception is thrown if any file in the vault cannot be removed.

10.4.12 `MteVault.dirExists` (Swift)

```
internal func dirExists(_ dir: String) -> Bool
```

Returns `true` if the directory exists, `false` if not. The `FileManager.default` API is used. Override to provide an alternate implementation.

`dir`: the directory name.

10.4.13 `MteVault.fileExists` (Swift)

```
internal func fileExists(_ dir: String, _ file: String) -> Bool
```

Returns `true` if the `file` exists in `dir`, `false` if not. The `FileManager.default` API is used. Override to provide an alternate implementation.

`dir`: the directory name.
`file`: the filename.

10.4.14 `MteVault.listFiles` (Swift)

```
internal func listFiles(_ dir: String) throws -> [String]
```

Returns a list of file basenames in a directory. Throws an exception on failure. The `FileManager.default` API is used. Override to provide an alternate implementation.

`dir`: the directory name.

10.4.15 `MteVault.createDir` (Swift)

```
internal func createDir(_ dir: String) throws
```

Creates a directory, including any intermediate directories as necessary. Throws an exception on failure. The `FileManager.default` API is used. Override to provide an alternate implementation.

`dir`: the directory name.

10.4.16 `MteVault.readFile` (Swift)

```
internal func readFile(_ dir: String, _ file: String) throws -> [UInt8]
```

Reads a file. Returns the file contents. Throws an exception on failure. The `Data` API is used. Override to provide an alternate implementation.

`dir`: the directory name.
`file`: the filename.

10.4.17 `MteVault.writeFile` (Swift)

```
internal func writeFile(_ dir: String, _ file: String, _ contents: [UInt8]) throws
```

Writes a file. Throws an exception on failure. The `Data` API is used. Override to provide an alternate implementation.

`dir`: the directory name.
`file`: the filename.
`contents`: the contents of the file to write.

10.4.18 `MteVault.removeDir` (Swift)

```
internal func removeDir(_ dir: String) throws
```

Removes a directory. Throws an exception on failure. The `java.io.File` API is used. Override to provide an alternate implementation.

Removes a directory. Throws an exception on failure. The `FileManager.default` API is used. Override to provide an alternate implementation.

`dir`: the directory name.

10.4.19 `MteVault.removeFile` (Swift)

```
internal func removeFile(_ dir: String, _ file: String) throws
```

Removes a file. Throws an exception on failure. The `FileManager.default` API is used. Override to provide an alternate implementation.

`dir`: the directory name.
`file`: the filename.

10.4.20 `MteVault.getTimestamp` (Swift)

```
internal func getTimestamp() -> UInt64
```

Returns the timestamp, byte swapped to increase the entropy in the upper bytes. The `Date().timeIntervalSince1970` function is used. Override to provide an alternate implementation. It is recommended to byte swap the value so the most-changing part of the timestamp is in the upper bytes of the `UInt64`.

11 Contact Eclypses

For more information, please contact:

info@eclypses.com

www.eclypses.com

+1.719.323.6680

All trademarks of Eclypses Inc. may not be used without Eclypses Inc.'s prior written consent. No license for any use thereof has been granted without express written consent. Any unauthorized use thereof may violate copyright laws, trademark laws, privacy and publicity laws and communications regulations and statutes. The names, images and likeness of the Eclypses logo, along with all representations thereof, are valuable intellectual property assets of Eclypses, Inc. Accordingly, no party or parties, without the prior written consent of Eclypses, Inc., (which may be withheld in Eclypses' sole discretion), use or permit the use of any of the Eclypses trademarked names or logos of Eclypses, Inc. for any purpose other than as part of the address for the Premises, or use or permit the use of, for any purpose whatsoever, any image or rendering of, or any design based on, the exterior appearance or profile of the Eclypses trademarks and or logo(s).